SLR210: Correctness of Synod

Recall that Synod implements *obstruction-free consensus* exporting one operation, *propose*, that takes an *value* in some *value set* V as an argument and returns a value in V or a special value *abort* $\notin V$ as a response. If an operation invoked by process p returns $v \in V$, we say that p *decides* v. If an operation returns *abort*, we say that p *aborts*. The abstraction is *long-lived*: a process can invoke the *propose* operation as many times as it wants.

The abstraction provides the following guarantees:

Validity Every decided value is a proposed value.

Agreement No two processes decide differently.

OF-Termination

1. If a correct process p proposes, it eventually decides or aborts.
2. If a correct process decides, no correct process aborts infinitely often.
3. If there is a time after which a single correct process p proposes a value sufficiently many times, p eventually decides.

> Notice that here the term *obstruction-freedom* is used in a slightly different way, compared to the conventional shared-memory definition that guarantees progress under the assumption that exactly one process is taking steps of from some point on. Here we guarantee progress under the assumption that exactly one process *proposes* from some point on, while still expecting that all other correct processes keep taking steps of the algorithm (without proposing). In particular they are expected to respond to request messages sent by the proposer.

The pseudocode of the Synod algorithm is presented in Algorithm 1.

To propose a value a process p_i selects the next *distinct* ballot number (line 2) and asks a majority of processes to accept or reject it (line 6). If some process has seen a larger or equal ballot number (either in the *read* phase or in the *impose* phase), the *propose* operation aborts (line 14). Otherwise, we check if some process in the majority has a decision estimate with a positive *impose ballot* number (line 18) and if so, the process selects the value with the highest impose ballot number as its *proposal* (line 19).

Then p_i issues an *impose* request (line 22). Again, if some process responds by saying that it has seen a higher read or impose ballot number, the operation aborts. Otherwise, p_i sends its proposal to all and decides (line 33).

Validity. The Validity property follows immediately from the algorithm: only proposed value can be decided.

Agreement. To prove the Agreement property, suppose that p_i is the process to decide a value v with the *lowest* associated ballot number b. Notice that every process has its own (disjoint) set of ballot numbers to choose, so such a process is well-defined.

Let p_j be the process that sends a $[IMPOSE, b', v']$ message (line 22) such that b' is the lowest ballot number higher than b attached to an *IMPOSE* message in the considered execution. We are going to show that v' must be v. By induction, this will imply that no two processes decide different values in any execution of our algorithm, as before deciding on a value v' with ballot b', p_j must have previously sent the corresponding *IMPOSE* message.

By the algorithm, before deciding, p_i has successfully completed its *impose* phase, and a majority of processes must have previously adopted p_i's impose ballot number b (line 27). As p_j, before sending an *IMPOSE* message with impose ballot b', has successfully completed its *read* phase, a majority of processes must have previously accepted p_j's read ballot number b' (line 11). Recall that, by our assumption, $b' > b$.

Let p_k be any process in the intersection of these two majorities. As $b < b'$, p_k must have received the impose request from p_i *before* the read request from p_j. Indeed, otherwise p_k would not accept the impose request from p_i with ballot number b as it previously adopted a higher ballot number b'. Thus, p_i must have received a *GATHER* response from p_k containing (b, v) as $(imposeballot, estimate)$ (line 16). As b' is the lowest ballot higher than b attached to an *IMPOSE* message, no process in the majority responding to p_j's read request can send a *GATHER* message containing (b'', v'') such that $b < b'' < b'$. Thus, p_j will select v as its proposal for the *impose* phase (line 19). Hence, $v = v'$ and we have the Agreement property.

OF-Termination. Part (1) of the OF-Termination property follows directly from the fact that a majority of processes are correct: eventually, every read or impose request reaches a majority of processes that will respond by sending either an *ABORT* message (resulting in aborting the *propose* operation) or a "meaningful" response) resulting in deciding.

Since a correct process sends its decision value to all before deciding (line 32), we also have part (2) of the OF-Termination property.

Finally, if there is a time after which, there is a single process p_i to propose, then, eventually, no process will see a read or impose ballot equal or higher to the ballot numbers issued by p_i (lines 8 and 24). Indeed, from some point on, p_i will be the only process in the system to increase its ballot numbers (line 2). Thus, eventually p_i will complete both read and impose phases of the *propose* operation and decide (line 33), which implies part (3) of OF-Termination.

Algorithm 1 Synod: code for process p_i

```
Local variables:
    ballot := i − n; proposal := nil; readballot := 0;
    imposeballot := i − n; estimate := nil; states := [nil, 0]^n

Upon propose(v) // process p_i proposes v:
    proposal := v; ballot := ballot + n; states := [nil, 0]^n
    send [READ, ballot] to all

Upon received [READ, ballot'] from p_j:
    if readballot > ballot' or imposeballot > ballot' then
        send [ABORT, ballot'] to p_j // reject the ballot number
    else
        readballot := ballot'
        send [GATHER, ballot', imposeballot, estimate] to p_j // accept the ballot number

Upon received [ABORT, ballot] from some process:
    return abort

Upon received [GATHER, ballot, estballot, est] from p_j:
    states[p_j] := [estballot, est]

Upon |states| > n/2: // received a majority of responses
    if ∃states[p_k] = [estballot, est] with estballot > 0 then
        select states[p_k] = [est, estballot] with highest estballot
        proposal := est // choose a potentially decided value
    states := [nil, 0]^n
    send [IMPOSE, ballot, proposal] to all

Upon received [IMPOSE, ballot', v] from p_j :
    if readballot > ballot' or imposeballot > ballot' then
        send [ABORT, ballot'] to p_j
    else
        estimate := v; imposeballot := ballot'
        send [ACK, ballot'] to p_j

Upon received [ACK, ballot] from a majority:
    send [DECIDE, proposal] to all

Upon receive [DECIDE, v]:
    send [DECIDE, v] to all
    return v
```